

Lucas Augusto da Silva Cardoso (202200332300)

DCA3703 | Programação Paralela | Relato de experimento

TAREFA 8: MONTE CARLO, OPENMP E EFEITOS DE MEMÓRIA

Este documento detalha a abordagem metodológica e a solução implementada para o Experimento Computacional proposto na disciplina de Programação Paralela, em 01/09/2026.

1. EXPERIMENTO

Implemente estimativa estocástica de "π" usando "rand()" para gerar os pontos. Cada thread deve usar uma variável privada para contar os acertos e acumular o total em uma variável global com "#pragma omp critical". Depois, implemente uma segunda versão em que cada thread escreve seus acertos em uma posição distinta de um vetor compartilhado. A acumulação deve ser feita em um laço serial após a região paralela. Compare o tempo de execução das duas versões. Em seguida, substitua "rand()" por "rand_r()" em ambas e compare novamente. Explique o comportamento dos quatro programas com base na coerência de cache e nos efeitos do falso compartilhamento.

2. CONCEITOS RELEVANTES

- **Falso compartilhamento:** é um problema de desempenho que ocorre em sistemas multiprocessados (como processadores *multicore*) quando diferentes *threads* tentam modificar variáveis independentes que estão armazenadas na mesma linha de cache (*cache line*).
- **Thread-Safe:** é um conceito de programação aplicada a códigos, funções ou estruturas de dados que garante seu funcionamento correto quando executados por múltiplas threads simultaneamente, sem causar

comportamentos imprevisíveis, corrupção de dados ou condições de corrida (race conditions).

- **Bit-sujo:** é um marcador de controle de 1 bit associado a um bloco de memória, presente em linhas de memória cache ou em páginas da memória virtual (tabela de páginas). Sua função é sinalizar se aquele bloco de dados foi alterado pelo processador após ter sido carregado da memória principal ou do disco.
- **Bit-inválido:** é um marcador de controle de 1 bit associado a cada bloco da memória cache ou a cada entrada da Tabela de Páginas na memória virtual. Sua função principal é indicar se o conteúdo carregado naquele bloco de memória contém dados válidos e úteis para o processador ou se deve ser ignorado.

3. MÁQUINA UTILIZADA

Para a execução deste experimento, estou utilizando uma máquina equipada com um processador i7 de 13ª geração (16 núcleos físicos e 24 threads), que é o componente central para a avaliação das diretivas OpenMP. O sistema dispõe também de 16 GB de memória RAM e um SSD de 512GB, garantindo um ambiente isolado de gargalos de alocação de memória ou paginação pelo sistema operacional, o que assegura a precisão das métricas de tempo.

4. CÓDIGO

O conjunto de códigos implementa a estimativa estocástica do valor de “pi” utilizando o Método de Monte Carlo paralelizado com a biblioteca OpenMP, comparando diferentes estratégias de sincronização e geração de números aleatórios. Nas quatro versões apresentadas, as threads dividem a geração de pontos bidimensionais “(x, y)” para calcular a proporção dos que caem no interior de um círculo de raio unitário; contudo, elas se diferenciam no mecanismo de consolidação dos acertos, alternando entre uma variável acumuladora global protegida por #pragma omp critical ao final da região paralela (Formatos 1 e 3) e a escrita dos acertos individuais em posições exclusivas de um vetor compartilhado processado sequencialmente (Formatos 2 e 4), e na função geradora de

pseudoaleatórios, contrastando o uso da rotina padrão rand() (não reentrante, que causa contenção de estado global) com a versão thread-safe “rand_r()” acoplada a sementes locais por thread.

Figura 1: Formato 1 (rand + critical)

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <math.h>
#include <omp.h>

#define PI 3.14159265358979323846

int main(int argc, char *argv[]) {
    long long total_pontos = 10000000LL; // 10 milhões de pontos
    if (argc > 1) {
        total_pontos = atoll(argv[1]);
        if (total_pontos <= 0) {
            fprintf(stderr, "Erro: O numero de pontos deve ser positivo.\n");
            return 1;
        }
    }

    printf("Threads ativas : %d\n", omp_get_max_threads());
    printf("Total de pontos: %lld\n", total_pontos);

    long long pontos_no_circulo_total = 0; // Compartilhada entre todas as threads
    long long contador_local;              // Será tornada 'private' para cada thread

    srand((unsigned int)time(NULL));

    double t_inicio = omp_get_wtime();

    // 1. Região paralela: define o que é compartilhado e o que é privado
    #pragma omp parallel \
        shared(total_pontos, pontos_no_circulo_total) \
        private(contador_local)
    {
        // 'contador_local' é private, portanto inicia com LIXO DE MEMÓRIA.
        // inicializando manualmente aqui dentro:
```

```
    contador_local = 0;

    double x, y, dist_sq; // Variáveis locais à thread (privadas)

    // Distribuindo as iterações do loop entre as threads
    #pragma omp for private(x, y, dist_sq)
    for (long long i = 0; i < total_pontos; i++) {
        x = (double)rand() / (double)RAND_MAX;
        // RAND_MAX é a constante máxima para rand(), gerando um número entre 0 e
1
        y = (double)rand() / (double)RAND_MAX;
        dist_sq = x * x + y * y;

        if (dist_sq <= 1.0) {
            contador_local++; // Incremento local sem condição de corrida
        }
    }

    // Atualizando o total global apenas uma única vez por thread
    #pragma omp critical
    {
        pontos_no_circulo_total += contador_local;
    }
  }

  double t_fim = omp_get_wtime();
  double tempo_total = t_fim - t_inicio;

  double pi_estimado = 4.0 * ((double)pontos_no_circulo_total / (double)total_pontos);
  double erro_absoluto = fabs(pi_estimado - PI);

  printf("Pontos no circulo : %lld / %lld\n", pontos_no_circulo_total, total_pontos);
  printf("Pi Estimado       : %.10f \n", pi_estimado);
  printf("Pi Real (PI)      : %.10f\n", PI);
  printf("Erro Absoluto     : %.10e\n", erro_absoluto);
  printf("Tempo de Execucao : %.6f segundos\n", tempo_total);

  return 0;
}
```

Figura 2: Formato 2 (rand + vetor)

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <math.h>
#include <omp.h>

#define PI 3.14159265358979323846

int main(int argc, char *argv[]) {
    long long total_pontos = 10000000LL; // 10 milhões de pontos
    if (argc > 1) {
        total_pontos = atoll(argv[1]);
        if (total_pontos <= 0) {
            fprintf(stderr, "Erro: O numero de pontos deve ser positivo.\n");
            return 1;
        }
    }

    int max_threads = omp_get_max_threads();
    printf("Threads ativas : %d\n", max_threads);
    printf("Total de pontos: %lld\n", total_pontos);

    // Vetor compartilhado onde cada thread armazenará seus acertos
    long long *acertos = (long long *)calloc(max_threads, sizeof(long long));
    if (acertos == NULL) {
        fprintf(stderr, "Erro ao alocar memoria para o vetor de acertos.\n");
        return 1;
    }

    long long pontos_no_circulo_total = 0;
    long long contador_local; // Privada para cada thread

    srand((unsigned int)time(NULL));

    double t_inicio = omp_get_wtime();

    // Cada thread escreve na sua posição do vetor compartilhado
    #pragma omp parallel \
        shared(total_pontos, acertos) \
        private(contador_local)
```

```
    {
        int tid = omp_get_thread_num();
        contador_local = 0;

        double x, y, dist_sq; // Variáveis locais à thread (privadas)

        // Distribuindo as iterações do loop entre as threads
        #pragma omp for private(x, y, dist_sq)
        for (long long i = 0; i < total_pontos; i++) {
            x = (double)rand() / (double)RAND_MAX;
            y = (double)rand() / (double)RAND_MAX;
            dist_sq = x * x + y * y;

            if (dist_sq <= 1.0) {
                contador_local++; // Incremento local sem conflito de cache
            }
        }

        // Cada thread escreve seus acertos em uma posição distinta do vetor
compartilhado
        acertos[tid] = contador_local;
    }

    // Acumulação em um laço serial após a região paralela
    for (int t = 0; t < max_threads; t++) {
        pontos_no_circulo_total += acertos[t];
    }

    double t_fim = omp_get_wtime();
    double tempo_total = t_fim - t_inicio;

    double pi_estimado = 4.0 * ((double)pontos_no_circulo_total / (double)total_pontos);
    double erro_absoluto = fabs(pi_estimado - PI);

    printf("Pontos no circulo : %lld / %lld\n", pontos_no_circulo_total, total_pontos);
    printf("Pi Estimado       : %.10f\n", pi_estimado);
    printf("Pi Real (PI)      : %.10f\n", PI);
    printf("Erro Absoluto     : %.10e\n", erro_absoluto);
    printf("Tempo de Execucao : %.6f segundos\n", tempo_total);
```

```
    free(acertos);

    return 0;
}
```

Figura 3: Formato 3 (rand_r + critical)

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <math.h>
#include <omp.h>

#define PI 3.14159265358979323846

int main(int argc, char *argv[]) {
    long long total_pontos = 10000000LL; // 10 milhões de pontos
    if (argc > 1) {
        total_pontos = atoll(argv[1]);
        if (total_pontos <= 0) {
            fprintf(stderr, "Erro: O numero de pontos deve ser positivo.\n");
            return 1;
        }
    }

    printf("Threads ativas : %d\n", omp_get_max_threads());
    printf("Total de pontos: %lld\n", total_pontos);

    long long pontos_no_circulo_total = 0; // Compartilhada entre todas as threads
    long long contador_local;              // Será tornada 'private' para cada thread

    double t_inicio = omp_get_wtime();

    // 1. Região paralela: define o que é compartilhado e o que é privado
    #pragma omp parallel \
        shared(total_pontos, pontos_no_circulo_total) \
        private(contador_local)
    {
        int tid = omp_get_thread_num();
        unsigned int seed = (unsigned int)time(NULL) + tid;
```

```
        // 'contador_local' é private, portanto inicia com LIXO DE MEMÓRIA.
        // inicializando manualmente aqui dentro:
        contador_local = 0;

        double x, y, dist_sq; // Variáveis locais à thread (privadas)

        // Distribuindo as iterações do loop entre as threads
        #pragma omp for private(x, y, dist_sq)
        for (long long i = 0; i < total_pontos; i++) {
            x = (double)rand_r(&seed) / (double)RAND_MAX;
            y = (double)rand_r(&seed) / (double)RAND_MAX;
            dist_sq = x * x + y * y;

            if (dist_sq <= 1.0) {
                contador_local++; // Incremento local sem condição de corrida
            }
        }

        // Atualizando o total global apenas uma única vez por thread
        #pragma omp critical
        {
            pontos_no_circulo_total += contador_local;
        }
    }

    double t_fim = omp_get_wtime();
    double tempo_total = t_fim - t_inicio;

    double pi_estimado = 4.0 * ((double)pontos_no_circulo_total / (double)total_pontos);
    double erro_absoluto = fabs(pi_estimado - PI);

    printf("Pontos no circulo : %lld / %lld (Correto!)\n", pontos_no_circulo_total,
total_pontos);
    printf("Pi Estimado       : %.10f (Correto!)\n", pi_estimado);
    printf("Pi Real (PI)      : %.10f\n", PI);
    printf("Erro Absoluto     : %.10e\n", erro_absoluto);
    printf("Tempo de Execucao : %.6f segundos\n", tempo_total);

    return 0;
}
```

Figura 4: Formato 4 (rand_r + vetor)

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <math.h>
#include <omp.h>

#define PI 3.14159265358979323846

int main(int argc, char *argv[]) {
    long long total_pontos = 10000000LL; // 10 milhões de pontos
    if (argc > 1) {
        total_pontos = atoll(argv[1]);
        if (total_pontos <= 0) {
            fprintf(stderr, "Erro: O numero de pontos deve ser positivo.\n");
            return 1;
        }
    }

    int max_threads = omp_get_max_threads();
    printf("Threads ativas : %d\n", max_threads);
    printf("Total de pontos: %lld\n", total_pontos);

    // Vetor compartilhado onde cada thread armazenará seus acertos
    long long *acertos = (long long *)calloc(max_threads, sizeof(long long));
    if (acertos == NULL) {
        fprintf(stderr, "Erro ao alocar memoria para o vetor de acertos.\n");
        return 1;
    }

    long long pontos_no_circulo_total = 0;
    long long contador_local; // Privada para cada thread

    double t_inicio = omp_get_wtime();

    // Cada thread escreve na sua posição do vetor compartilhado
    #pragma omp parallel \
        shared(total_pontos, acertos) \
        private(contador_local)
    {
```

```
        int tid = omp_get_thread_num();
        unsigned int seed = (unsigned int)time(NULL) + tid;
        contador_local = 0;

        double x, y, dist_sq; // Variáveis locais à thread (privadas)

        // Distribuindo as iterações do loop entre as threads
        #pragma omp for private(x, y, dist_sq)
        for (long long i = 0; i < total_pontos; i++) {
            x = (double)rand_r(&seed) / (double)RAND_MAX;
            y = (double)rand_r(&seed) / (double)RAND_MAX;
            dist_sq = x * x + y * y;

            if (dist_sq <= 1.0) {
                contador_local++; // Incremento local sem conflito de cache
            }
        }

        // Cada thread escreve seus acertos em uma posição distinta do vetor
compartilhado
        acertos[tid] = contador_local;
    }

    // Acumulação em um laço serial após a região paralela
    for (int t = 0; t < max_threads; t++) {
        pontos_no_circulo_total += acertos[t];
    }

    double t_fim = omp_get_wtime();
    double tempo_total = t_fim - t_inicio;

    double pi_estimado = 4.0 * ((double)pontos_no_circulo_total / (double)total_pontos);
    double erro_absoluto = fabs(pi_estimado - PI);

    printf("Pontos no circulo : %lld / %lld (Correto!)\n", pontos_no_circulo_total,
total_pontos);
    printf("Pi Estimado       : %.10f (Correto!)\n", pi_estimado);
    printf("Pi Real (PI)      : %.10f\n", PI);
    printf("Erro Absoluto     : %.10e\n", erro_absoluto);
    printf("Tempo de Execucao : %.6f segundos\n", tempo_total);
```

```
    free(acertos);

    return 0;
}
```

4.1 AJUSTE DO CÓDIGO PARA OBSERVAÇÃO

O código dos formatos 2 e 4 foi reestruturado para que o incremento seja realizado diretamente na posição correspondente do vetor compartilhado (acertos[tid]++) durante a execução do laço paralelo, dispensando o uso de acumuladores locais privados (contador_local). Essa alteração foi feita propositalmente para permitir a observação empírica dos efeitos do falso compartilhamento (“false sharing”). Uma vez que os elementos consecutivos do vetor residem na mesma linha de cache de 64 bytes da arquitetura, as escritas frequentes e concorrentes efetuadas por núcleos distintos forçam o protocolo de coerência a invalidar continuamente essas linhas em todas as caches locais (cache bouncing), gerando um tráfego excessivo de barramento e um aumento expressivo no tempo de execução, o que evidencia na prática a degradação de desempenho provocada por essa anomalia arquitetural.

Antes da alteração, a implementação dos formatos 2 e 4 utilizava uma estratégia baseada em acumulação privada com escrita única ao final. Cada thread possuía uma variável local independente, denominada “contador_local”, que era inicializada com zero antes do laço e tornada privada por meio da cláusula “private(contador_local)”. Ao longo de todas as milhões de iterações do laço paralelo, os acertos sorteados eram contabilizados exclusivamente nessa variável local, que residia na pilha privada da thread ou diretamente em registradores da CPU, sem que o vetor compartilhado acertos sofresse qualquer operação de leitura ou escrita. Somente após o término completo de todas as iterações da thread é que o valor acumulado era transferido, em uma única instrução de escrita, para a posição correspondente no vetor compartilhado (acertos[tid] = contador_local).

5. RESULTADOS

Abaixo, é possível visualizar o resultado da compilação dos códigos antes e depois da alteração.

Figura 5: Tempo em segundos da aplicação em função da quantidade de threads

Programa	2T	4T	6T	8T	10T	12T	14T	16T	18T	20T	22T	24T
Formato 1 (rand + critical)	1,9713	0,9172	0,8748	0,8638	0,93276	1,15465	1,2755	1,31028	1,37547	1,41023	1,30407	1,27796
Formato 2 (rand + vetor)	1,6734	0,9154	0,8794	0,8989	0,9583	1,2464	1,35756	1,42270	1,48749	1,44503	1,39362	1,38927
Formato 3 (rand_r + critical)	0,0255	0,01330	0,00959	0,0079	0,0067	0,0058	0,0060	0,01531	0,0072	0,00686	0,00727	0,0074
Formato 4 (rand_r + vetor)	0,0488	0,03222	0,03024	0,0356	0,0333	0,0308	0,0301	0,03909	0,0240	0,022	0,02162	0,02238

Figura 6: Formato 1 (rand + critical) x Formato 2 (rand + vetor)

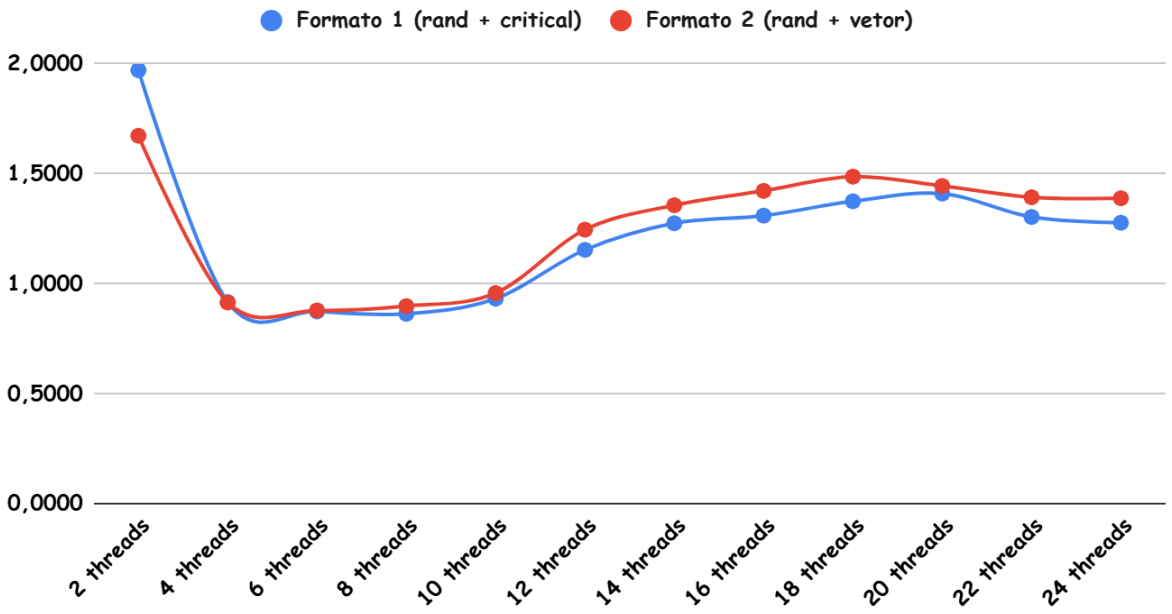
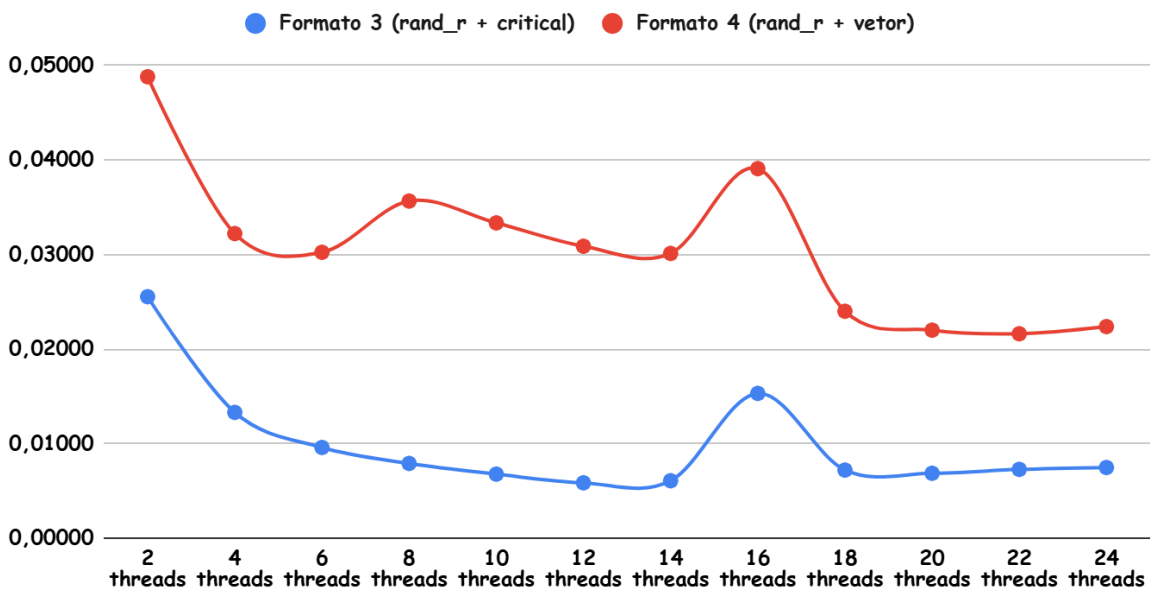


Figura 6: Formato 3 (rand_r + critical) x Formato 4 (rand_r + vetor)



A análise dos resultados demonstra com clareza como a escolha das funções de biblioteca e o padrão de acesso à memória afetam a escalabilidade do código. O ponto de maior impacto no desempenho foi a transição da função padrão “rand()” para a versão reentrante “rand_r()”, resultando em um ganho de velocidade de até 200 vezes. Isso ocorre porque a “rand()” original utiliza um estado interno global protegido por “locks”, o que faz com que todas as “threads” disputem o mesmo recurso a cada iteração, gerando gargalos severos à medida que o número de “threads” aumenta. Em contrapartida, a “rand_r()” utiliza sementes locais por *”hread”, permitindo que a geração de números aleatórios ocorra de maneira totalmente independente e paralela.

No que tange aos mecanismos de consolidação dos acertos, a abordagem que utilizou a instrução “#pragma omp critical” no encerramento da região paralela (Formato 3) apresentou o melhor desempenho geral, atingindo a marca de 0,0058 segundos em 12 “threads”. Já a estratégia de escrita em um vetor compartilhado (Formato 4) foi substancialmente mais lenta em razão do fenômeno de falso compartilhamento (“false sharing”). Como os elementos do vetor alocado sequencialmente ocupam a mesma linha de cache (“cache line”), a atualização frequente da posição de uma “thread” inválida constantemente a cache das demais, forçando recargas desnecessárias e onerando o protocolo de coerência do processador. Por fim, nota-se que o tempo de execução diminui consistentemente

até a faixa de 8 a 12 “threads”, momento a partir do qual os ganhos estabilizam ao atingir o limite dos núcleos físicos do sistema.

6. CONSIDERAÇÕES FINAIS

O experimento evidenciou a importância de alinhar a programação paralela às características da arquitetura de hardware subjacente. A simples distribuição de tarefas via diretivas de paralelismo não garante ganho de desempenho caso existam gargalos de sincronização ou conflitos no subsistema de memória.

A substituição de rotinas não reentrantes (“rand()”) por funções ‘thread-safe’ (“rand_r()”) provou ser o fator decisivo para viabilizar a escalabilidade da aplicação, eliminando a contenção por recursos globais. Além disso, a comparação entre a consolidação final via “#pragma omp critical” e a escrita em vetor compartilhado destacou o impacto negativo do falso compartilhamento (“false sharing”) sobre a hierarquia de cache. Em suma, o Formato 3 (“rand_r” com redução ao final) consolidou-se como a arquitetura ideal para o problema de Monte Carlo, combinando independência total de execução entre as *threads* com um uso eficiente da memória cache.